

Sécurité des Réseaux Mobiles

Android

- ▶ Android repose fondamentalement sur le noyau Linux. Ce noyau fournit les fonctionnalités de base du système (gestion de la mémoire, des processus, du réseau et les pilotes matériels) sur lesquelles les couches supérieures d'Android viennent s'appuyer.
- ▶ Historiquement, le noyau Linux a été testé, éprouvé et sécurisé dans des millions de serveurs avant d'arriver sur mobile. Google a adapté ce noyau pour Android en y ajoutant des mécanismes spécifiques aux contraintes des smartphones (comme une gestion différente de l'énergie et le système de communication inter-processus appelé Binder).

Architecture d' Android



Librairies de base

Libc: fournit des services minimaux d'**interface avec le système d'exploitation** (par exemple open, malloc, printf, etc.)

SSL: gère la **sécurité des communications réseau** (couche transport).

Surface Manager: gère la **composition** à l'écran du **graphique** rendu par différents processus.

SGL: permet l'affichage de **graphique 2D**, c'est la base graphique de la plupart des applications.

OpenGL|ES 1.x: permet l'affichage de **graphique 3D** (une implémentation logicielle est fournie, elle peut être accélérée par le matériel).



Librairies de base

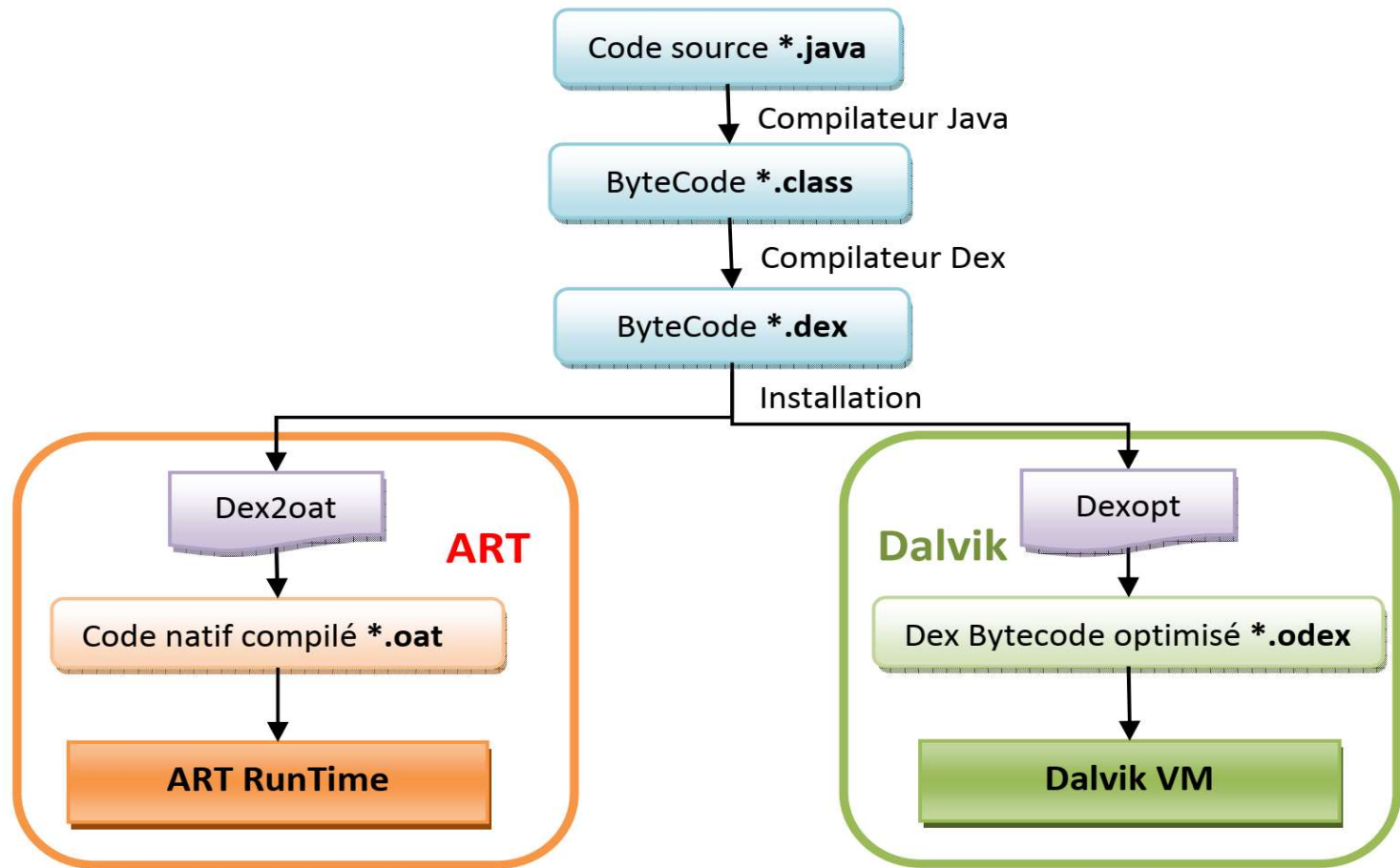
FreeType: permet le **rendu typographique** de qualité.

WebKit: permet le *layout et le rendu de pages HTML*, ainsi que l'**exécution** du langage **javascript**

Media Framework: fournit le support pour le décodage d'un ensemble de **formats audio et vidéo** (MPEG 4, H.264, MP3, AAC, etc.)

SQLite: fournit le service de **stockage de données** au sein d'une base de données.

Moteur d'exécution (Android RunTime)



Moteur d'exécution (Android RunTime)

- Dalvik et ART sont des machines virtuelles qui permettent aux applications de s'exécuter sur un dispositif mobile Android, indépendamment des différences matérielles et quelque soit le modèle.
- la différence majeure entre Dalvik et ART se situe sur le moment d'interprétation du code.
- Dalvik interprète le bytecode à la volée pour être exécuté (JIT :Just in Time). Alors que ART compile le bytecode en avance (au moment d'installation) une fois pour toutes, avant son utilisation (AOT : Ahead-Of-time).

Application framework

- Les fonctionnalités fournis par les bibliothèques d'Android sont ensuite utilisées par cette couche sous forme de bibliothèques Java
- 1. **Les services cœurs de la plateforme** (Core Platform Services) : Un service est une application, qui s'exécute en arrière-plan et qui n'a aucune interaction avec l'utilisateur
 - **Activity Manager** : il prend en charge de la gestion du cycle de vie des applications ainsi que le maintien de la pile de navigation qui permet d'aller d'une application à une autre.
 - **Package Manager** : il est utilisé, par l'Activity Manager, pour charger les informations provenant des fichiers (.apk) (Android package file).
 - **Window Manager** : il s'occupe de la gestion des fenêtres des applications (quelle fenêtre doit être affichée devant une autre à l'écran).

Application framework

- ▶ **Resource Manager** : Il s'intéresse pour la gestion de tous ce qui n'est pas du code, toutes les ressources (images, fichier audio, etc.).
- ▶ **Content Provider** : Il gère le partage de données entre applications.
- ▶ **View System** : Il fournit tous les composants graphiques : listes, grilles, boutons, etc...

Application framework

2. Les services matériels (Hardware Services) : ce type de service fournit un accès vers les API matérielles :

- *Telephony Service* : il permet d'accéder aux interfaces téléphoniques (GSM, 3G, etc.).
- *Location Service* : il permet d'accéder au GPS.
- *Bluetooth Service* : il permet d'accéder à l'interface Bluetooth.
- *Wi-Fi Service* : il permet d'accéder à l'interface Wifi.
- *USB Service* : il permet d'accéder aux interfaces USB.
- *Sensor Service* : il permet d'accéder aux capteurs (capteur de luminosité, de proximité, d'accélération, etc.).

Menaces de Sécurité Mobile

Les appareils mobiles sont exposés à un large spectre de menaces en raison de leur omniprésence, de leur connexion permanente aux réseaux et de la quantité de données sensibles qu'ils traitent.

- **Logiciels malveillants (Malware & Spyware)** : des logiciels malveillants peuvent voler des données, suivre les activités de l'utilisateur ou transmettre des informations sensibles à des tiers sans consentement.
- **Attaques de phishing** : les attaquants envoient des messages trompeurs (e-mails, SMS) imitant des entités légitimes pour soutirer des identifiants ou des données financières.
- **Cryptojacking mobile** : des pirates détournent la puissance de calcul de l'appareil pour miner des cryptomonnaies en arrière-plan, dégradant les performances et augmentant la consommation d'énergie.

Menaces de Sécurité Mobile

- ▶ **Interception sur réseau radio** : les réseaux mobiles (Wi-Fi, Bluetooth, NFC) présentent des faiblesses permettant d'intercepter ou d'altérer les données échangées, voire de déployer des logiciels espions
- ▶ **Applications malveillantes** : applications contenant du code malveillant qui contournent les contrôles d'accès du système d'exploitation.

Mécanismes de sécurité d'Android

La sécurisation d'une plate-forme ouverte nécessite une architecture de sécurité robuste et des programmes de sécurité rigoureux.

1. UID Linux pour les applications normales

- Dans un système Linux traditionnel (comme Ubuntu), plusieurs applications sont exécutées par un même utilisateur physique. Sous Android, la donne change : chaque application est considérée comme un utilisateur Linux distinct.
- Lorsqu'une application est installée, le système lui attribue automatiquement un identifiant utilisateur unique (**UID**). L'application s'exécute dans un processus distinct avec cet UID spécifique.

Mécanismes de sécurité d'Android

2. Sandboxing d'application

- ▶ grâce aux permissions classiques du système de fichiers Linux (lecture, écriture, exécution), le noyau empêche physiquement une application d'accéder aux fichiers ou à la mémoire d'une autre application. C'est ce qu'on appelle le Sandboxing applicatif ou bac à sable.
- ▶ Un Sandbox est une zone isolée du système qui n'a pas accès au reste des ressources du système, à moins que les permissions d'accès ne soient explicitement accordées par l'utilisateur lorsque l'application est installée.

Mécanismes de sécurité d'Android

3. Inter Process Communication (IPC)

- Puisque les applications sont isolées dans des bacs à sable étanches, elles ne peuvent pas communiquer directement entre elles. Pour pallier cela, Android a introduit **Binder**, un module spécifique ajouté au noyau Linux. Binder remplace l'IPC traditionnel de Linux et permet aux applications d'échanger des données de manière strictement contrôlée et sécurisée au niveau du noyau.

4. Permissions

- Android fournit un système avec un ensemble prédéfini de permissions pour certaines tâches que l'application peut demander. Par exemple, si vous souhaitez que votre application utilise la caméra d'un téléphone, vous devez demander la permission `android.permission.CAMERA`.

Mécanismes de sécurité d'Android

5. Processus de signature et de publication

- Après le développement de l'application, l'étape suivante consiste à la publier et à la partager avec d'autres. Cependant, les applications ne peuvent pas simplement être ajoutées à une boutique en ligne(ex : google play) et partagées, elles doivent d'abord être signées.
- La signature cryptographique sert de marque vérifiable placée par le développeur de l'application. Il identifie l'auteur de l'application et garantit que l'application n'a pas été modifiée depuis sa distribution initiale

Introduction à SELinux sous Android

- Security-Enhanced Linux (SELinux) est un système de Contrôle d'Accès Obligatoire (MAC — **Mandatory Access Control**) intégré au noyau Linux.
- Développé à l'origine par la NSA pour sécuriser les systèmes d'exploitation, il a été intégré progressivement par Google dans Android depuis la version 4.3 (Jelly Bean) et activé de manière stricte depuis Android 5.0 (Lollipop).
- Dans Android, SELinux agit comme une surcouche de sécurité par-dessus les permissions classiques de Linux (le DAC — *Discretionary Access Control*). Il limite considérablement les dommages potentiels causés par des failles de sécurité, des applications malveillantes ou des processus système compromis en confinant strictement ce que chaque processus a le droit de faire.

Introduction à SELinux sous Android

- Security-Enhanced Linux (SELinux) est un système de Contrôle d'Accès Obligatoire (MAC — **Mandatory Access Control**) intégré au noyau Linux.
- Développé à l'origine par la NSA pour sécuriser les systèmes d'exploitation, il a été intégré progressivement par Google dans Android depuis la version 4.3 (Jelly Bean) et activé de manière stricte depuis Android 5.0 (Lollipop).
- Dans Android, SELinux agit comme une surcouche de sécurité par-dessus les permissions classiques de Linux (le DAC — *Discretionary Access Control*). Il limite considérablement les dommages potentiels causés par des failles de sécurité, des applications malveillantes ou des processus système compromis en confinant strictement ce que chaque processus a le droit de faire.

Les Concepts Fondamentaux de SELinux

- ▶ Android utilise SELinux pour appliquer un Contrôle d'Accès Obligatoire (MAC - Mandatory Access Control) sur tous les processus. SELinux fonctionne selon le principe du refus par défaut : si une action n'est pas explicitement autorisée par la politique de sécurité dictée par l'administrateur (ici, Google/le constructeur), elle est bloquée.

Les Modes de Fonctionnement de SELinux

Sur un appareil Android, SELinux peut fonctionner selon :

- ▶ **Enforcing (Application stricte)** : C'est le mode par défaut et obligatoire en production. La politique de sécurité est strictement appliquée. Si une application tente une action non autorisée, le noyau la bloque immédiatement et génère une erreur d'autorisation (EPERM) tout en l'inscrivant dans les logs.
- ▶ **Permissive (Audit seul)** : Dans ce mode, SELinux n'applique pas la politique, il ne bloque aucune action. Cependant, il enregistre dans les journaux système toutes les actions qui *auraient* dû être bloquées en mode Enforcing. Ce mode est exclusivement utilisé par les développeurs et les constructeurs lors de la phase de création de l'appareil pour déboguer et écrire de nouvelles règles.

L'impact de SELinux sur le "Root"

- L'intégration d'un SELinux strict a complètement changé le paysage du modding et du "Root" sur Android. Auparavant, obtenir les privilèges de superutilisateur (su) suffisait pour avoir un contrôle total sur l'appareil.
- Aujourd'hui, même si un processus obtient l'UID 0 (Root) via une faille de sécurité, s'il se trouve dans le domaine **untrusted_app**, SELinux continuera de lui interdire de modifier les fichiers vitaux du système.
- Pour rooter un appareil Android moderne (via Magisk par exemple), il faut non seulement obtenir les droits su, mais aussi patcher la politique SELinux en mémoire pour contourner le Contrôle d'Accès Obligatoire

Autres fonctionnalités de sécurité

Voici les "Autres fonctionnalités" de sécurité essentielles au niveau du kernel mobile :

1. **Seccomp-BPF** (Secure Computing Mode)

- Sur un appareil Android, une application normale n'a pas besoin de parler à l'intégralité du noyau Linux. Par exemple, une application de calculatrice n'a pas besoin d'utiliser les commandes système liées à la gestion des cartes réseaux ou à l'horloge système matérielle.
- **Le rôle de Seccomp** : Ce mécanisme permet de filtrer et de réduire drastiquement le nombre d'**appels système (syscalls)** qu'une application a le droit de faire au noyau.

Autres fonctionnalités de sécurité

Les Namespaces (Espaces de noms)

- ▶ Les namespaces sont la technologie fondatrice des conteneurs (comme Docker), et ils sont massivement utilisés sur Android pour isoler les applications.
- ▶ Le noyau Linux utilise les espaces de noms pour "mentir" aux applications. Par exemple, avec le **Mount Namespace** (espace de noms de montage), chaque application voit une arborescence de fichiers système complètement différente.
- ▶ Une application Android croit qu'elle est seule sur le téléphone, car le noyau Linux lui cache l'existence des autres applications et de leurs fichiers.

Autres fonctionnalités de sécurité

3. **ASLR et KASLR** (Address Space Layout Randomization)

- Il s'agit d'une technique de défense contre l'exploitation de failles de corruption de mémoire.
- **ASLR** (pour les applications) et **KASLR** (pour le Kernel lui-même) consistent à mélanger aléatoirement l'emplacement du code et des données dans la mémoire vive (RAM) à chaque démarrage du téléphone.
- **L'avantage** : Si un hacker écrit un script pour attaquer une adresse mémoire spécifique où se trouve un mot de passe (attaque de type "buffer overflow"), son attaque échouera car la cible aura changé de place.

Autres fonctionnalités de sécurité

4. **dm-verity** (Device Mapper Verity)

- Implémenté directement dans le sous-système de blocs du noyau Linux, dm-verity est le moteur du **Verified Boot** (Démarrage validé).
- Ce composant du noyau calcule en temps réel, de manière cryptographique, l'intégrité de la partition système pendant que le téléphone est allumé.
- Si un malware très avancé parvient à devenir Root et tente de modifier un fichier système caché, dm-verity au niveau du noyau le détectera instantanément car le hachage du fichier ne correspondra plus à l'arbre cryptographique. Le noyau bloquera alors la lecture du fichier pour protéger le système.

Sécurité au niveau Dalvik (et ART)

Permissions au niveau Dalvik

- Au lieu de laisser l'application accéder librement aux ressources du téléphone (caméra, SMS, contacts), Android impose un modèle de permissions déclaratives : Toute permission requise doit être déclarée dans le fichier **AndroidManifest.xml** de l'application via la balise **<uses-permission>**.

Ces permissions sont réparties en plusieurs niveaux de protection :

1. **Normal** : Opérations sans risque majeur (ex: vibrer, accès internet basique). Accordées automatiquement à l'installation.
2. **Dangerous** : Accès à des données privées (contacts, localisation, micro). L'utilisateur doit explicitement donner son accord (via un pop-up d'autorisation).

Sécurité au niveau Dalvik (et ART)

3. **Signature** : Permission accordée uniquement si l'application qui la demande est signée par le même certificat que l'application qui déclare la permission (très utilisé pour la communication sécurisée entre deux applications d'un même développeur).
4. **SignatureOrSystem** : Réservé aux applications signées par le constructeur ou installées dans la partition système.

Sécurité au niveau Dalvik (et ART)

Signature du code Dalvik

Android exige que **tous** les fichiers APK soient signés numériquement avec un certificat avant de pouvoir être installés.

- ▶ Le développeur signe son code (compilé en fichier classes.dex) avec sa propre clé privée. Le certificat public est inclus dans le dossier **META-INF/** de l'APK.

Le rôle de la signature (Confiance et Intégrité)

1. **L'intégrité du bytecode (.dex)** : Dans la structure du fichier .dex, on trouve une somme de contrôle (checksum Adler32) et un hachage cryptographique (SHA-1). La signature garantit que le code compilé n'a pas été altéré ou infecté par un malware entre le moment où le développeur a créé l'application et le moment où elle est installée sur votre téléphone.

Sécurité au niveau Dalvik (et ART)

2. **La confiance temporelle (Trust on First Use)** : C'est le point le plus important. Quand vous installez "WhatsApp" pour la première fois, Android enregistre le certificat de cette application. Lorsque WhatsApp propose une mise à jour, le système vérifie que le nouvel APK est signé **avec la même clé privée** que le premier. Si un pirate crée un faux "WhatsApp" avec un autre certificat, Dalvik refusera formellement de l'installer en tant que mise à jour. Cela empêche le vol d'identité d'applications.
3. **Le partage d'UID (Sandbox sharing)** : En temps normal, chaque application a un bac à sable (UID) séparé. Mais si un développeur signe deux applications avec la même clé privée et demande le paramètre *sharedUserId*, Dalvik autorisera ces deux applications à s'exécuter dans le même processus Linux, leur permettant de partager librement leur mémoire et leurs fichiers.

Sécurité au niveau Utilisateur

- **L'écran de verrouillage** (Lock screen) : C'est la première ligne de défense contre l'accès physique non autorisé. Android propose plusieurs méthodes :
 - **Authentification classique** : Code PIN (recommandé à 6 chiffres minimum), Mot de passe alphanumérique ou Schéma (Pattern).
 - **Biométrie** : Empreinte digitale, reconnaissance faciale ou d'iris.

- **Gestion des clés secrètes et privées (Keystore)**

Pour éviter que les applications ne stockent des mots de passe ou des clés de chiffrement en clair dans leur code, Android propose l'**Android Keystore**.

- Ce système permet de générer et de stocker des clés cryptographiques dans un environnement matériellement sécurisé (comme un *Trusted Execution Environment* - TEE)
- Une fois la clé dans le Keystore, l'application peut demander au système de chiffrer ou signer des données avec cette clé, mais **le matériel refuse d'exporter la clé elle-même**. Ainsi, même si le téléphone est rooté ou compromis, les clés restent inaccessibles aux attaquants.

Sécurité au niveau Utilisateur

► Gestion des certificats

Android gère une infrastructure à clés publiques (PKI) avec un magasin de certificats (Certificate Store).

- Il contient les certificats racine de confiance (Root CA) préinstallés par le système pour valider les connexions HTTPS.
- Les utilisateurs et les administrateurs d'entreprise peuvent installer des certificats personnalisés (User CA) pour se connecter à des réseaux Wi-Fi d'entreprise (EAP-TLS) ou des VPN.

Sécurité du Stockage

La protection des données au repos est cruciale en cas de perte ou de vol de l'appareil.

- **Chiffrement des données** (Data encryption): Le chiffrement sur Android a évolué en deux phases majeures :
 1. **FDE (Full Disk Encryption)** : Utilisé d'Android 5.0 à 9.0. Il chiffre l'intégralité de la partition de données (/data) avec une clé unique dérivée du mot de passe de l'utilisateur.
 2. **FBE (File-Based Encryption)** : Obligatoire depuis Android 10. Le chiffrement est appliqué fichier par fichier, avec des clés différentes. Cela a permis d'introduire le **Direct Boot** : certains fichiers (comme ceux de l'application d'alarme ou de téléphonie) peuvent être déchiffrés par le système dès le démarrage, avant même que l'utilisateur n'ait tapé son code PIN, tandis que les données privées restent verrouillées cryptographiquement.

Sécurité du Stockage

► Démarrage Sécurisé (Secure Boot / Verified Boot)

Pour s'assurer que le système d'exploitation n'a pas été altéré par un malware complexe (comme un rootkit), Android implémente le processus de **Verified Boot** (Démarrage validé).

- À l'allumage, le matériel (Hardware Root of Trust) vérifie la signature cryptographique du *Bootloader* (le programme d'amorçage).
- Le *Bootloader* vérifie ensuite la signature du noyau Linux.
- Le noyau vérifie la signature de la partition système d'Android.

C'est une chaîne de confiance : si un seul élément a été modifié ou infecté, le téléphone refusera de démarrer, affichera un message d'avertissement rouge et protégera ainsi les données de l'utilisateur contre une compromission silencieuse.